AI 5200 Project 1 Report

David Everett

Part 1: Final Models

The original baseline used 30 residual blocks, stream width 128, hidden width 256, tanh, pre-LayerNorm, Xavier initialization, and a two-layer classifier. It trained with plain SGD at learning rate 0.05 and batch size 256, without momentum, a scheduler, regularization, or augmentation. The baseline trained for 10 epochs on MNIST, 30 on CIFAR, and 10 on ImageNet.

Architecture	**MNIST**	**CIFAR-10**	**ImageNet32**
Model	Flat residual MLP	Residual patch MLP	Residual patch MLP
Residual blocks	10	6 mixing blocks	12 mixing blocks
Stream width	392	128 per patch	128 per patch
Hidden width	1,024	Token 64; channel 256	Token 64; channel 256
Image grouping	784 pixel values	64 patches of 4x4 RGB pixels	64 patches of 4x4 RGB pixels
Activation	Two-piece Maxout	GELU	GELU
Normalization	BatchNorm	LayerNorm	LayerNorm
Classifier	Linear to 10 classes	Patch mean; 128 to 256 to 10 with GELU	Patch mean; linear 128 to 1,000

Every linear weight matrix starts with Xavier normal initialization and zero bias. In the patch models, normalization occurs before each residual branch and before averaging the patches. As a note, the MNIST settings still contain an activation value of GELU, but Maxout replaces the branch activations and the classifier is linear, so GELU is not used in that model's forward pass. Similarly, the nested patch settings determine the actual CIFAR and ImageNet depths; the unused flat-model settings do not describe those models.

Training recipe	**MNIST**	**CIFAR-10**	**ImageNet32**
Optimizer	AdamW	SGD; momentum 0.9	SGD; momentum 0.9

Training recipe	MNIST	CIFAR-10	ImageNet32
Initial learning rate	0.001	0.05	0.05
Epochs	40	90	20
Batch size	256	256	256
Schedule	Cosine toward 0.00001	Cosine toward 0.00001	Cosine toward 0.00001
Weight decay	0.01	0	0
Branch dropout	0.1	0.1	0
Augmentation	Rotation up to 7 degrees; shifts up to 2 pixels; black padding	Horizontal flip probability 0.5; shifts up to 2 pixels with reflection; RGB jitter 0.05	Horizontal flip probability 0.5; noise standard deviation 0.02; RGB jitter 0.05

CIFAR's MLP classifier has no dropout. Neither RGB model uses rotation in its final recipe, and ImageNet doesn't use translation. The momentum entry in the MNIST dictionary is inactive because that task uses AdamW.

Task	Baseline accuracy	Best recorded accuracy	Gain in points	Recorded training time
MNIST	95.95%	99.27%	3.32	Baseline 1.1 min; best 2.5 min on local GPU
CIFAR-10	48.62%	78.11%	29.49	Baseline 4.8 min local; best 4.1 min on HPC
ImageNet32	6.77%	25.42%	18.65	Baseline 37.2 min local; best 37.0 min on HPC

Part 2: What Mattered Most

Processing Images as Patches

The patch configuration produced my largest improvement. For example, CIFAR Change 15 improved the best accepted flat-model accuracy from 63.12% to 71.71%, a gain of 8.59 percentage points; the previous trial scored 62.40%, making that change a direct increase of 9.31 points. ImageNet Change 1, enabling the patch architecture, improved its original 6.77% baseline to 13.54%.

A 4x4 RGB patch contains 48 values, and the shared projection converts each patch into 128 learned features. Token mixing combines information between patch positions, and channel mixing combines features within a patch. This provided the models a useful way to process local groups, but a flat model still retains positions through its input indices, so flattening did not erase all spatial information.

Making the earlier flat model wider and shallower

Before using patches, CIFAR Change 2 increased width from 128 to 512 and hidden width from 256 to 1,024, reduced depth from 30 to 8, introduced SwiGLU branches, and used a linear classifier. Accuracy improved from 48.62% to 55.10%. This was a 6.48-point gain, but several settings changed together to say that SwiGLU caused it by itself.

Change 3 then replaced SwiGLU with two-piece Maxout and improved 55.10% to 57.68%. Both models reached nearly zero training loss while their test accuracy remained much lower, meaning they could fit the training images but still needed a better way to generalize. So, I experimented more with augmentation and eventually patch processing.

Replacing Tanh with GELU

ImageNet Change 3 improved accuracy from 17.29% to 21.42%, a gain of 4.13 points, while final training loss decreased from 4.1471 to 3.7395. I chose GELU because tanh saturates to 1 for large inputs, whereas GELU doesn't cap large positive outputs. The result supported keeping GELU. However, I did not measure saturation in the earlier tanh trial, so its lower performance alone doesn't prove that saturation caused the problem.

Adding Momentum

ImageNet Change 2 increased SGD momentum from zero to 0.9 and improved 13.54% to 17.29%. Its training loss similarly decreased from 4.5708 to 4.1471. Momentum combines recent gradient directions with the current gradient, which helped the model make more progress within the same number of epochs.

CIFAR Change 7 also improved accuracy, from 59.35% to 63.12%. Its augmented training loss increased from 0.4660 to 0.5293, so momentum did not simply lower every loss value. This reinforces the concept that test data performance matters more than selecting the smallest training loss.

Choosing Useful Augmentation and Regularization

CIFAR Change 17 added dropout and improved accuracy from 71.71% to 72.76%. Change 18 removed rotation and noise together, improving accuracy to 75.45%. These

changes suggest that regularization helped, but some image transformations were making the small images harder to learn from without adding useful variation. Removing the two transformations at once obscures which one contributed more to the decreased accuracy.

MNIST benefited from shifts because Change 25 improved accuracy from 98.82% to 99.01%, finally crossing the 99% threshold. Later rotation helped reach its best of 99.27%. The final RGB jitter trials added smaller gains, from 77.82% to 78.11% on CIFAR and 24.80% to 25.05% on ImageNet. My RGB jitter slightly changes an image's red, green, and blue channel intensities during training to help the model recognize objects despite small differences in color and lighting.

Other changes helped finish the recipes. ImageNet Change 8 increased patch depth from 6 to 12 and improved accuracy from 21.99% to 23.26%. The first Change 9 doubled epochs from 10 to 20 and reached 24.80% accuracy. Its final cosine trial reached 25.42%. For CIFAR, Change 21 added an MLP classifier and improved 76.01% to 77.63%; increasing from 80 to 90 epochs in Change 22 reached 77.82% before RGB jitter.

Complete CIFAR-10 Recipe Comparisons

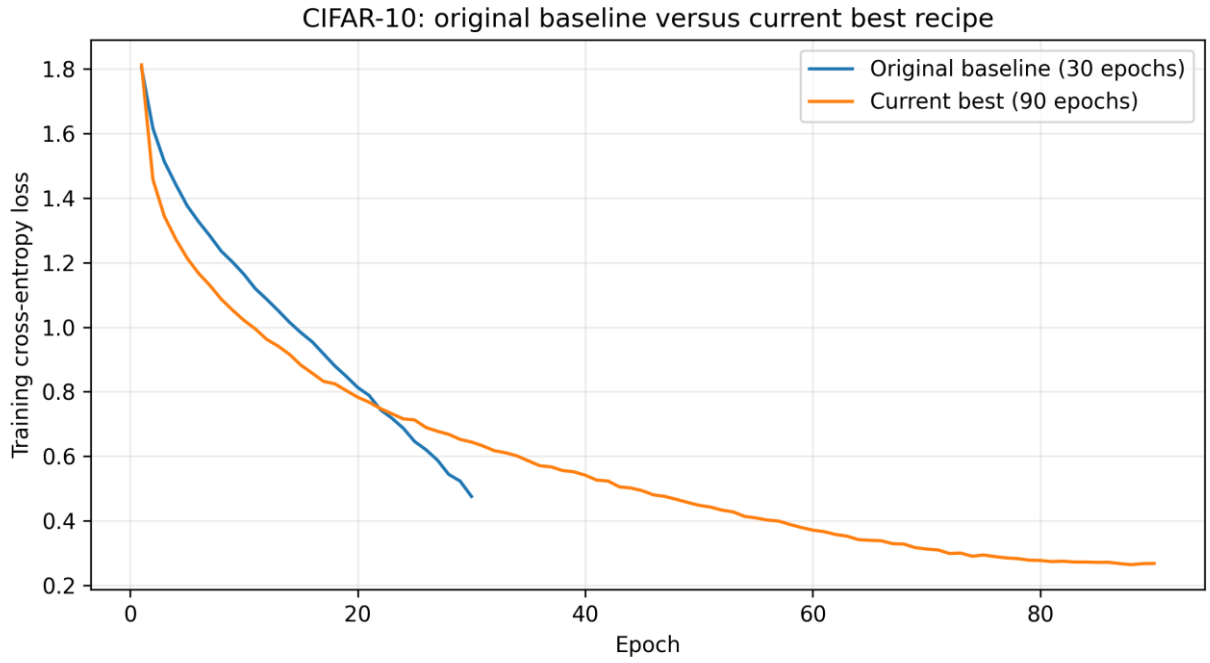


Figure 1. CIFAR-10 baseline model versus best model training loss

Both curves began near 1.8 loss. At epoch 10, the best recipe had lower loss, 1.0211 versus 1.1638. At epoch 30, the baseline was lower, 0.4750 versus 0.6438. The best recipe then continued to 0.2672 at epoch 90. Because this compares complete model differences

along with the training conditions given augmentation and dropout, the lower baseline loss at epoch 30 doesn't establish better generalization for the baseline.

Part 3: Training Dynamics and Ablation on ImageNet32 Model

How I Measured the Model

I used analysis.ipynb to train my final ImageNet architecture from scratch and measure what happened inside it. The notebook uses the same model settings as the harness, but holds out five training images per class for validation. This left 1,276,167 training images and 5,000 validation images. I reused the same clean batch of 128 other training images at initialization and after epochs 2, 10, and 20. All measurements used eval mode so changes came from training rather than different images or dropout.

For gradients and updates, the notebook copied the model and optimizer together. It calculated the fixed-batch loss with gradients enabled, took one real optimizer step on the copy, and discarded it. This preserved momentum without changing the model being trained. Measurements happened before the scheduler advanced, using the learning rate from the completed epoch. The notebook and saved CSV files make these measurements repeatable without adding analysis work to the graded training function.

Learning Curves

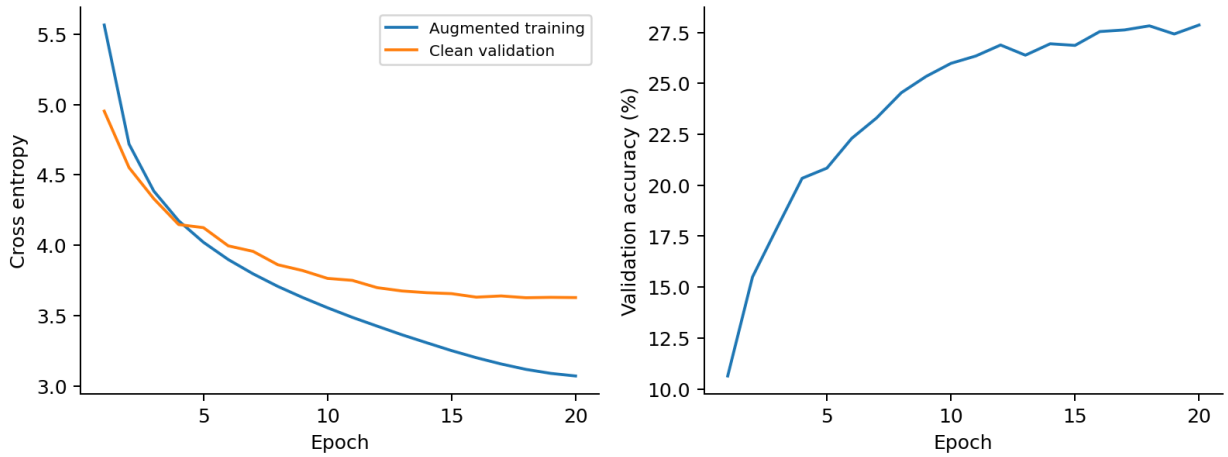


Figure 2. ImageNet analysis training loss, clean validation loss, and validation accuracy

On ImageNet, training loss decreased from 5.5648 at epoch 1 to 3.07 at epoch 20. Validation accuracy increased from 10.64% to 27.86%, while validation loss decreased from 4.9527 to 3.6273, and training took 35.69 minutes on the remote HPC cluster with one GPU. The 27.86% validation result uses this held-out training split, so it's separate from the 25.42% harness score.

Through the epochs, the model kept learning, but the later gains were smaller; validation accuracy was already 25.98% at epoch 10. Validation loss was lowest at epoch 18, at 3.6262, and was almost unchanged at the end. This suggests diminishing returns near the end rather than a sudden failure to generalize. Training loss uses augmented images and is averaged during updates, while validation is measured afterward on clean images, thus their difference includes more than overfitting alone.

Activation Variance

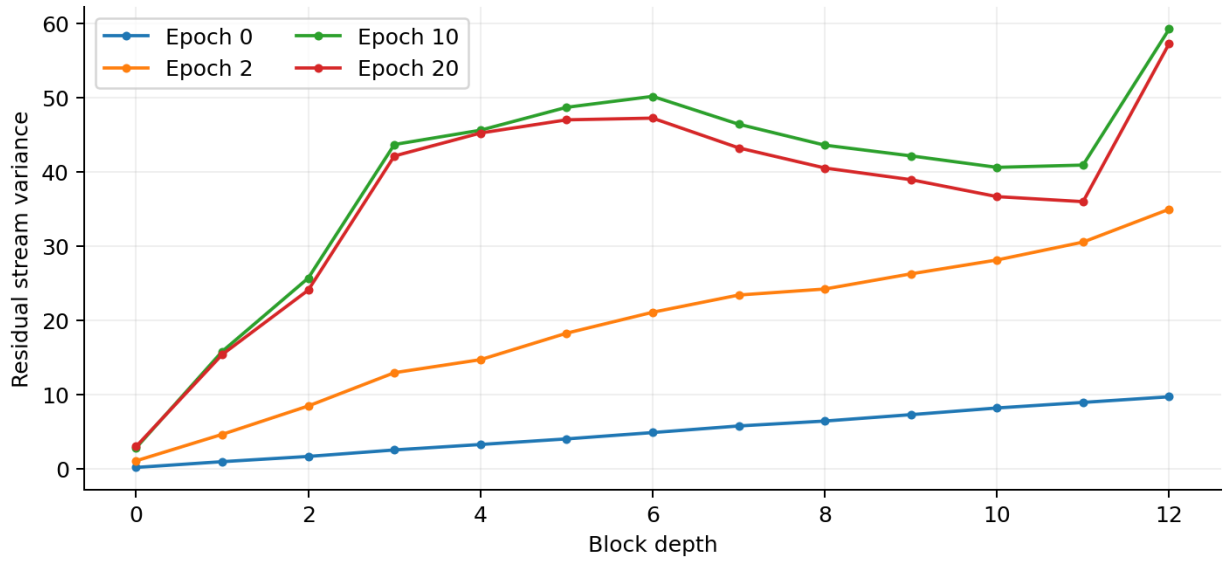


Figure 3. Residual stream variance after projection and each block at four checkpoints

Variance measures how widely values spread around their mean. I measured it across the examples, patches, and features after the patch projection and every complete block. Figure 3 shows that the residual stream grew with depth. At initialization, variance increased from 0.1616 after projection to 9.6826 after block 12. At epoch 20, it increased from 2.9792 to 57.2737, about 19.2 times the projection variance.

The final profile was not a steady increase. Variance reached 47.2295 after block 6, decreased to 35.9753 after block 11, and then increased in block 12. Residual corrections can strengthen or cancel parts of the existing signal. LayerNorm normalizes branch inputs, but it doesn't reset the accumulated stream to variance one. The signal grew, but the measured profiles and continued learning do not support calling this an uncontrolled numerical explosion.

Branch to Stream Ratio

$$Branch\ ratio\ = \frac{RMS(branch\ correction)}{RMS(branch\ input)}$$

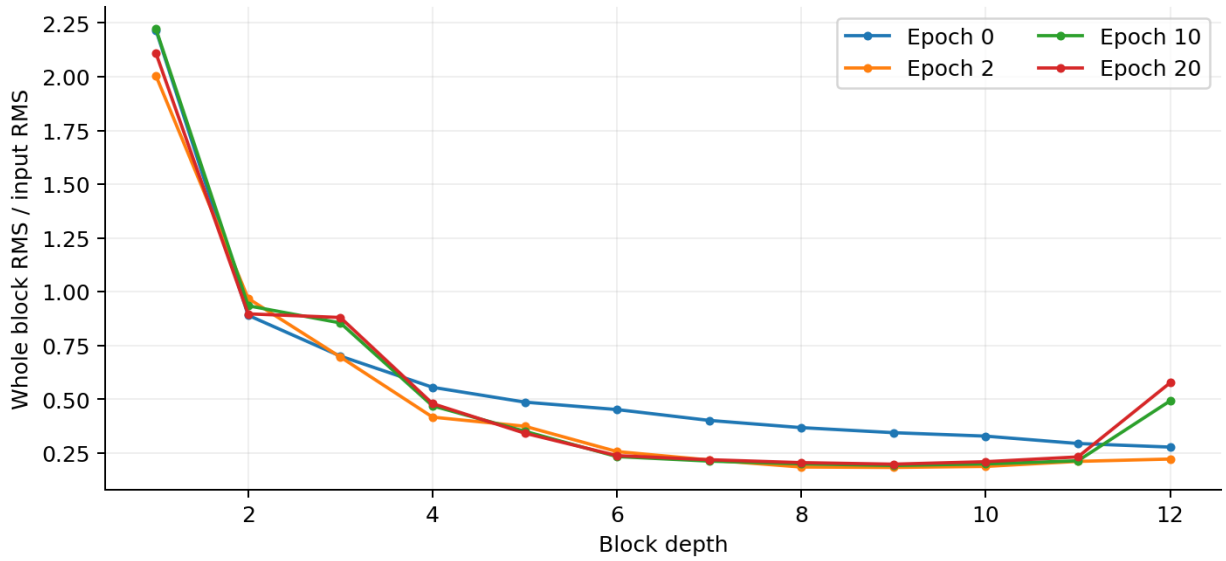


Figure 4. Whole-block correction RMS divided by the RMS of its input stream

RMS measures a signal's typical magnitude. The branch-to-stream ratio divides the correction's RMS by its input's RMS. I recorded token and channel branches separately, plus the complete block correction. The channel branch receives the stream after token mixing has already been added.

Figure 4 shows large early corrections and smaller corrections through much of the later network. At epoch 20, the whole-block ratio was 2.109 in block 1, 0.898 in block 2, and 0.199 in block 9. It rose again to 0.577 in block 12. The final token ratios in blocks 8–11 were about 0.106–0.133. These branches changed the stream relatively little, but their corrections were not zero. A falling ratio can also reflect a growing input stream, so I used ablation to check whether small corrections were actually unimportant.

RMS Gradients

$$RMS(g) = \sqrt{mean(g^2)}$$

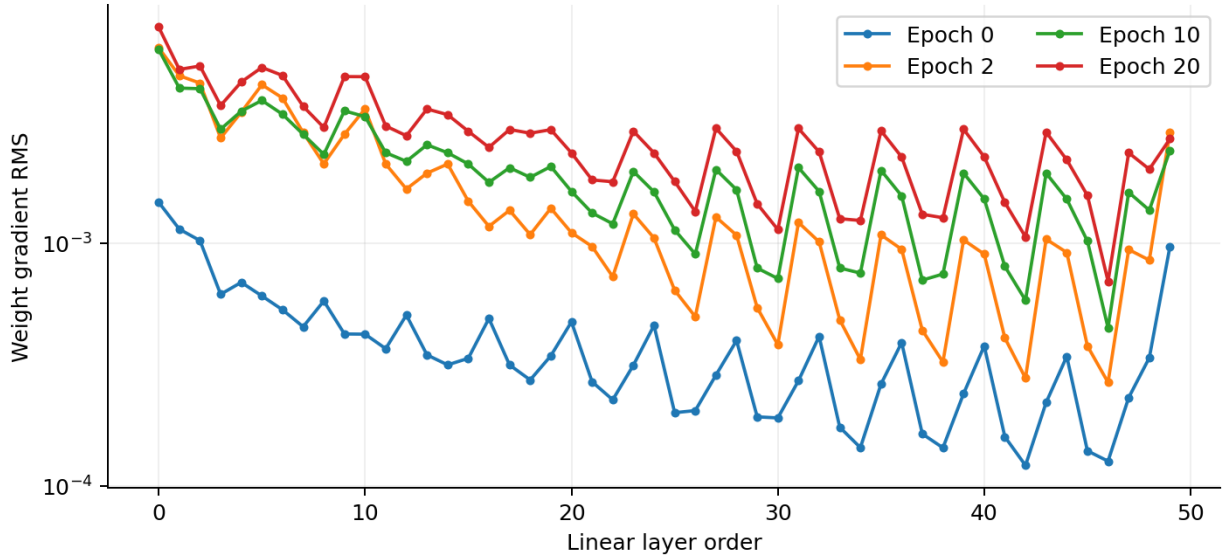


Figure 5. Weight gradient RMS for all 50 linear matrices at four checkpoints

I measured gradient RMS for all 50 linear weight matrices. Figure 5 uses linear-layer order: the projection comes first, followed by four matrices per mixing block and the classifier. Its horizontal axis is therefore not the block number.

All matrices had finite, nonzero gradients at all four checkpoints. At epoch 20, the largest gradient RMS was 0.007776 in the patch projection. The smallest was 0.000699 in the final token branch's output projection. Earlier layers generally had larger gradients, with repeated peaks between different matrix types. The measured gradients do not support an obvious vanishing or exploding gradient problem. Howver, this doesn’t mean every layer's scale was ideal, and four checkpoints cannot describe every training step.

Update to Weight Ratio

$$Update\ ratio\ = \frac{RMS(weights\ after\ step\ -\ weights\ before\ step)}{RMS(weights\ before\ step)}$$

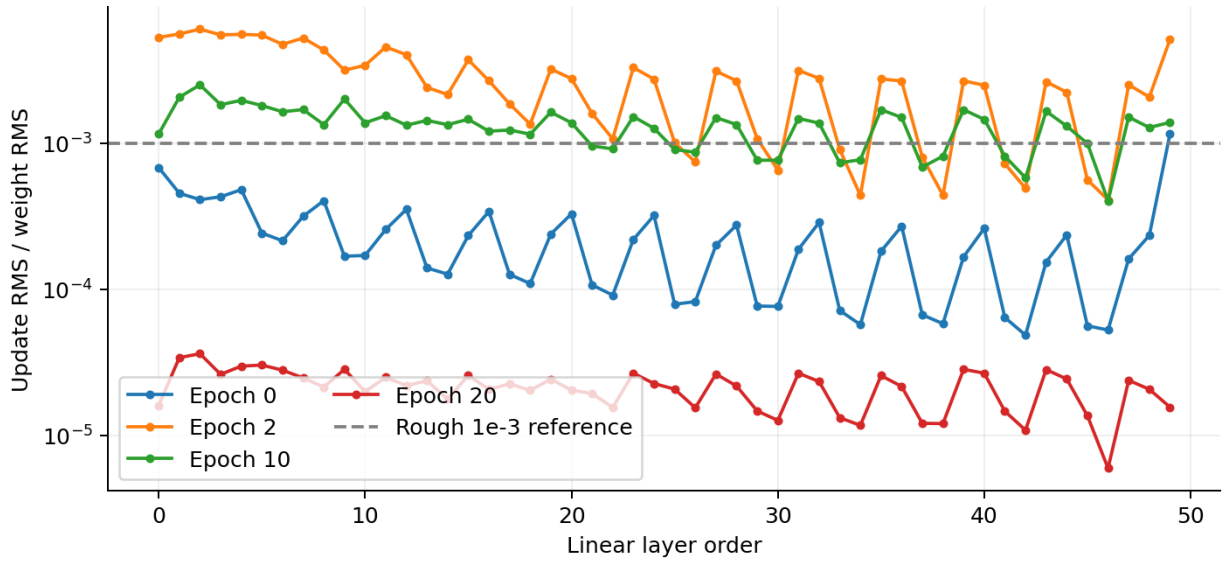


Figure 6. Actual diagnostic optimizer update RMS relative to weight RMS

This ratio divides the RMS weight change from one diagnostic optimizer step by the RMS weights before that step. A ratio of 0.001 means the update's RMS is one thousandth of the weights' RMS. Taking an actual copied optimizer step matters because momentum carries information from previous updates. Initialization has no accumulated momentum yet.

In Figure 6, update ratios ranged from about 0.000408 to 0.006022 at epoch 2. By epoch 20, they ranged from about 0.000006 to 0.000036. The small final ratios agree with the cosine schedule: the probe learning rate fell from 0.05 to 0.0003177, about 157 times smaller. Gradients had not disappeared. I would therefore not diagnose a bad learning rate just because the final ratios were below the approximate 0.001 reference. The final probe occurs before the scheduler's last step to 0.00001.

Saturation

My final ImageNet model uses GELU, so the requested tanh and ReLU saturation statistics do not apply.

Which Branches Mattered

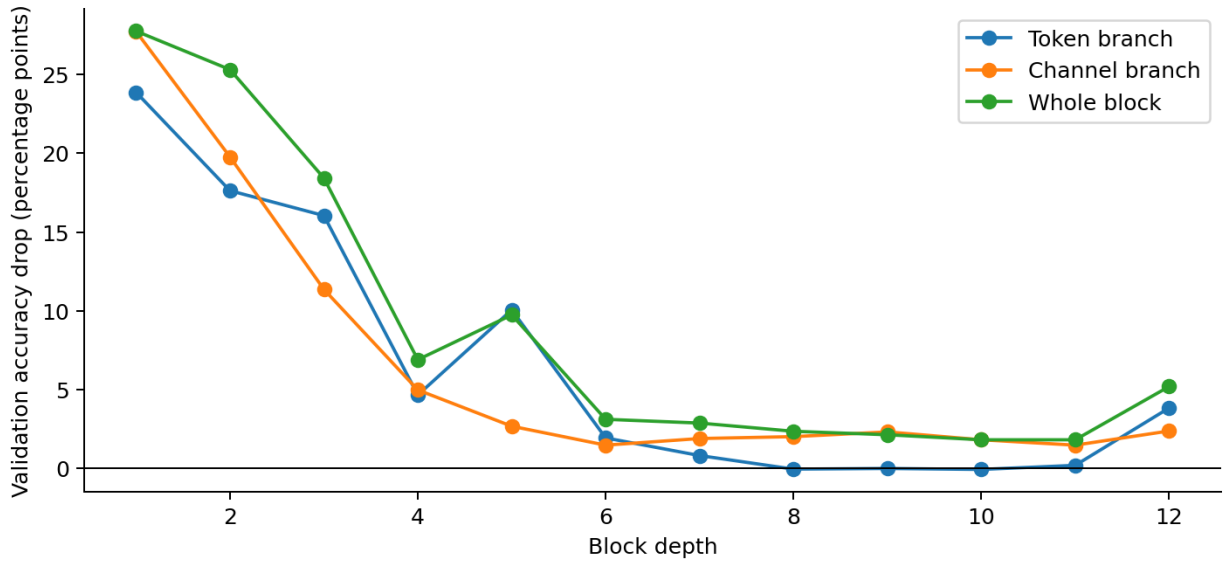


Figure 7. Validation accuracy drop after individual branch and whole-block removals

Each patch block has a token and channel correction, giving 24 individual branch tests. Returning a zero correction preserved its skip connection. I also bypassed each complete block by returning its input, giving the twelve whole-block points in Figure 7. Every removal was restored before the next test. Accuracy drop is the full model's 27.86% validation accuracy minus the ablated accuracy, expressed in percentage points.

The network relied heavily on its early blocks; for example, removing complete blocks 1, 2, and 3 reduced accuracy by 27.74, 25.30, and 18.42 percentage points. Bypassing block 1 left only 0.12% accuracy, close to the 0.1% accuracy of random guessing across 1,000 classes. In contrast, token branches 8 and 10 could be removed with tiny measured improvements of 0.04 and 0.06 points. These represent only two and three additional correct predictions out of 5,000, so they didn't provide a reliable improvement. Token branch 9 had zero net accuracy change, but lost 64 correct predictions and gained 64 others, so it still affected individual predictions. However, the deeper network was not entirely unnecessary. Removing whole blocks 10 or 11 still cost 1.82 points each, and removing block 12 cost 5.20 points, plus channel branches also retained useful contributions.

The ratios broadly agreed with ablation: larger whole-block corrections tended to produce larger accuracy drops, with a Pearson correlation of about 0.885 across these twelve blocks. Still the ratios and ablation didn't match perfectly: block 5 had a smaller ratio than block 4, but its removal caused a larger drop.

This study shows that the model doesn't rely on every block equally. Removing the first block reduced validation accuracy significantly, suggesting that it builds features that the rest of the network strongly depends on. Several later blocks had smaller effects, but the final block still caused a 5.20 percentage point drop when removed. This suggests that the network uses its depth unevenly: the early blocks establish useful representations, while later blocks refine them, with some contributing more than others. The branch-to-stream ratios generally supported this pattern because larger corrections tended to correspond to larger accuracy drops. However, correction size was not a perfect measure of importance. A small correction can still contain information that affects the final prediction. Some late token-mixing branches could be removed with almost no accuracy change, suggesting that additional mixing between image patches becomes less useful at those depths. Thus a more efficient model is definitely possible, but these findings do not establish the best depth.

Part 4: What Didn't Work

More depth did not always help. CIFAR's change 16 and 17 entries tested 12 and then 10 patch blocks. Twelve blocks reduced training loss from 0.3877 to 0.2529 but reduced accuracy from 71.71% to 70.06%. Ten blocks reached 70.88%, still below six. This is consistent with poorer generalization under that recipe. I returned to six blocks rather than assuming a larger model must perform better. I assume the depth caused overfitting or just generally unhelpful extra complexity.

CIFAR Change 9 replaced LayerNorm with BatchNorm. Training loss decreased from 0.5293 to 0.2308, but accuracy decreased from 63.12% to 62.22%. BatchNorm worked in the later MNIST Maxout recipe, so this trial did not show that either normalization method was always superior. It showed that the choice interacted with the task and model. My best explanation is that BatchNorm made this model more dependent on training-batch statistics, which may have transferred less well to unseen CIFAR images.

The MLP classifier also behaved differently across tasks. It helped CIFAR, but ImageNet's second Change 9 reduced accuracy from 24.80% to 23.92% and increased loss from 3.2406 to 3.2771. I restored ImageNet's linear classifier. The extra layer added capacity but did not improve this training recipe. The experiment doesn't establish whether the cause was optimization, generalization, or a need for different settings. My best explanation is that the extra nonlinear layer made optimization harder under the same training settings, since even the training loss became worse.

AdamW did not beat the tested momentum-SGD parents on CIFAR or ImageNet. CIFAR Change 8 reached 62.47% compared with 63.12%, and ImageNet Change 7 reached

20.84% compared with 21.99%. Both used AdamW at learning rate 0.001. These replacements were worse under the settings tested, though that doesn't AdamW is generally unsuitable since I didn't test many learning rates. For example it remained useful in the MNIST model. I suspect AdamW needed its own learning-rate and schedule tuning, while the existing recipe already worked well with momentum SGD.

Extra augmentation sometimes hurt. MNIST noise reduced its recorded best from 99.27% to 99.14%, while ImageNet translation changed 21.71% to 21.58%. Removing CIFAR rotation and noise improved its patch model. I assume some transformations may obscure details in already small images or need more training to become useful.

MNIST label smoothing changed 98.46% to 98.42%. Its loss of 0.2830 used smoothed target probabilities, so it should not be compared directly with ordinary cross-entropy. Reducing batch size to 64 reached 98.25% and increased training time to 4.9 minutes. I assume label smoothing added more regularization than this MNIST model needed, while the smaller batch made updates noisier and required more steps per epoch.

Part 5: Across the Three Tasks

MNIST has ten grayscale categories with fairly consistent digit structure. A flat MLP could use its 784 pixel inputs effectively. Small positional changes helped it recognize handwriting in different places and at slightly different angles. Its Maxout, BatchNorm, and linear-classifier combination worked well together. The best result was already close to perfect accuracy, leaving limited room for further gains.

CIFAR also has ten categories, but color objects, backgrounds, and positions vary more. The flat models could fit the training images closely while remaining much less accurate on unseen examples. Patch processing provided a more useful organization of image information. The final recipe uses six mixing blocks, dropout, shifts, horizontal flips, RGB jitter, an MLP classifier, and more training. Increasing patch depth hurt under the tested recipe, while improving the classifier helped.

ImageNet32 has 1,000 classes and many more training images. It must separate more varied and fine-grained categories from only 32x32 pixels. Twelve mixing blocks improved its result, but the extra classifier layer did not. I kept a linear classifier, no dropout, and mild flipping, color, and noise augmentation. Twenty ImageNet epochs also involve far more training examples than twenty CIFAR epochs, so epoch counts alone do not compare computation fairly.

These results show that more parameters were not enough by themselves. Shared patch processing and the training recipe mattered, and depth had to be justified by its

contribution. The ImageNet ablation supports the conclusion that early blocks were essential, while some later token branches were less useful individually, and the final block still mattered. This experiment demonstrates that MLP's way of processing image structure is a major constraint.

Part 6: Explanations Beyond the Basics

Patch Mixing and GELU

The patch model follows the basic MLP-Mixer idea from the paper: https://arxiv.org/abs/2105.01601. One shared linear projection embeds each patch. A token MLP mixes patch positions for each learned feature, while a channel MLP mixes features within each patch. Each branch adds a correction through a residual connection. After the last block, normalized patch features are averaged into one vector for classification. The channels inside these blocks are learned features, not just the original red, green, and blue channels. I used this structure because it improved image recognition while keeping the network within the allowed MLP building blocks. GELU smoothly weights an input according to its value. It reduces much of the negative range while allowing large positive values to continue increasing. I used this activation function because it was recommended in the MLP mixer paper.

Maxout

Two-piece Maxout learns two candidate values for each hidden feature and keeps the larger; I discovered this from this paper: https://proceedings.mlr.press/v28/goodfellow13.pdf. Its preceding linear layer therefore produces twice the resulting hidden width. This lets the model learn which candidate is more useful for each input. I used it because it adds more complexity to capture the patterns.

Image Transformations

Reflection padding uses nearby reflected image content at exposed borders, while MNIST uses black padding. These were used to fill the blanks; MNIST could use black padding since the edges likely wouldn't carry useful information, but the other models used reflection padding to hopefully preserve overall image patterns and variation. RGB jitter selects one multiplier between 0.95 and 1.05 for each image's red, green, and blue channel. A multiplier is shared across all pixels in that channel, changing color balance consistently. The code converts prepared images back to the 0-1 range before multiplying, then returns to -1 to 1. I used this because objects can appear under different lighting and color balance, so small changes to channel intensities may help the model recognize

useful patterns without depending too heavily on exact colors. Gaussian noise instead changes individual pixels independently. I tried this because small variation in pixel values might help the model gather information from blurred instances.